

Architettura degli Elaboratori

System calls, Procedure



SAPIENZA
UNIVERSITÀ DI ROMA

Alessandro Checco

alessandro.checco@uniroma1.it

alessandrochecco.github.io

Special thanks and credits:

Andrea Sterbini, Iacopo Masi,

Claudio di Ciccio

[S&PdC] Capitolo 2



Informazioni sull'esonero

- Quando: 9 Maggio 2025 dalle 16 alle 19
- Dove: Aula 16 e 17 laboratori di via Tiburtina
- Prova pratica di assembly in rars
- Note:
 - necessaria la registrazione al link seguente
 - aperto solo a chi sta frequentando le lezioni

Argomenti

Argomenti della lezione

- Recap su Vettori e Matrici
- System calls
- Procedure (subroutines usando jal)
- Esercitazione

Scaricate il simulatore RARS da:

<https://github.com/rarsm/rars/releases/download/latest/rars-flatlaf.jar>

Per l'esame useremo la versione originale (vecchia **ma con meno bug**)

https://github.com/TheThirdOne/rars/releases/download/v1.6/rars1_6.jar

- è scritto in Java (**gira su qualsiasi OS, su Windows serve jdk23**)
- è **estensibile** (diversi plug-in per esaminare il comportamento della CPU e della Memoria)
- è un IDE integrato:
 - **Editor con evidenziazione della sintassi** assembly ed **autocompletamento** delle istruzioni
 - **Help completo** (istruzioni, syscall ...)
 - **Simulatore** con possibilità di **esecuzione passo-passo** e uso di **breakpoint**
 - Permette l'**esecuzione da riga di comando** e la **compilazione di più file**

Vettori e Matrici



SAPIENZA
UNIVERSITÀ DI ROMA

Matrici: vettori di vettori

Una matrice $M \times N$ è altro una successione di M vettori, ciascuno di N elementi

- il numero di elementi totali è: $M \times N$
- la dimensione totale in byte è: $M \times N \times \text{dimensione_elemento}$
- la si definisce staticamente come un vettore contenente $M \times N$ elementi uguali

Matrice: **.word** **0:91** # spazio per una matrice 7×13 word

L'elemento **e**,
di coordinate
 $x=2, y=9$ si trova
ad una distanza di:
• 2 righe
• più 9 elementi
dall'inizio, ovvero ad
un offset di
 $2 \times 13 + 9 = 35$ word
cioè
 $35 \times 4 = 140$ byte

13 colonne

7 righe	0	1	2	3	4	5	6	7	8	9	10	11	12	0
	13	14	15	16	17	18	19	20	21	22	23	24	25	1
	26	27	28	29	30	31	32	33	34	e				2
														3
														4
														5
														6
	0	1	2	3	4	5	6	7	8	9	10	11	12	

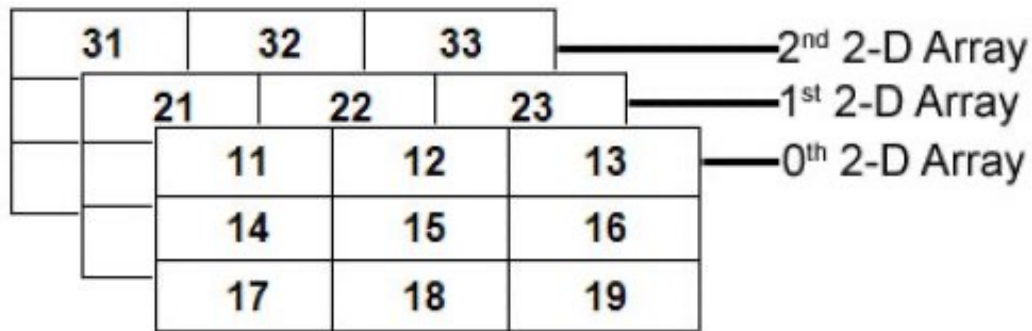
y

x

Matrici 3D

Questo è esattamente il calcolo svolto dal compilatore C, per esempio

scrivere	<code>matrice[x][y]</code>
per una matrice di interi definita	<code>int matrice[NUM_RIGHE][NUM_COL]</code>
equivale a usare l'indirizzo	<code>matrice + (x*NUM_COL+y)*sizeof(int)</code>



Matrici a 3 dimensioni

Una matrice 3D di **dimensioni** $L \times M \times N$ è una successione di L matrici 2D grandi $M \times N$

L'elemento a coordinate l, x, y è preceduto da:

l «strati» (matrici $M \times N$ formate da $M \cdot N$ elementi)

x «righe» di M elementi sullo stesso strato

y «elementi» sulla stessa riga e strato

Quindi l'elemento si trova a $l * (M * N) + x * N + y$ elementi dall'inizio della matrice 3D

e la sua posizione in memoria è `indirizzo_matrice + (l * (M * N) + x * N + y) * dim_el.`

Recap

Matrici: **Indirizzo + offset di riga** dall'indirizzo fornisce l'elemento che vogliamo ottenere

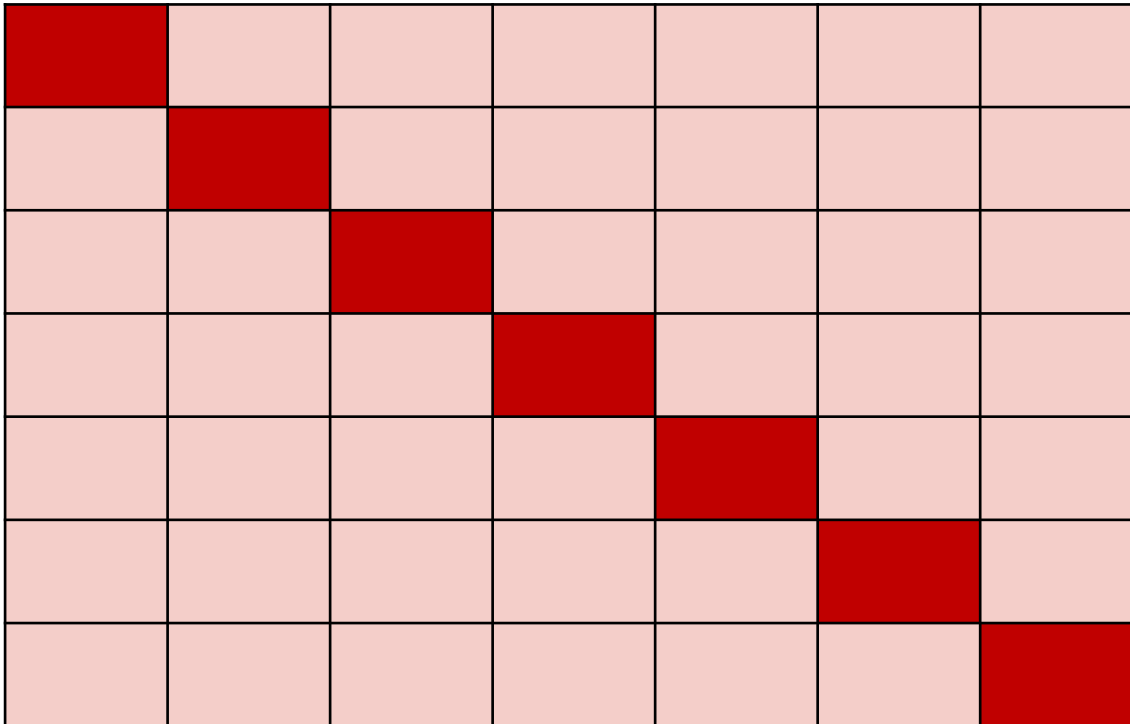
$$(x, y) \rightarrow \text{indirizzo}[0] + N * x + y \quad N = \#cols$$

Matrici 3D: simili a vettori ma dobbiamo tenere traccia degli strati anche.

Indirizzo + offset di strato + offset di colonna

$$(l, x, y) \rightarrow \text{indirizzo}[0] + [l * (N * M) + N * x + y]$$
$$N = \#col, M = \#rig$$

Somma della diagonale (INEFF.)



Somma della diagonale (INEFF.)

.data

matrice2D: **.word** 0:400 # matrice quadrata di 20x20 word

DIM: **.word** 20 # lato della matrice

.text

```
main:          li  t0, 0           # coordinata y (colonne)
               li  t1, 0           # coordinata x (righe)
               li  t2, 0           # somma iniziale
               lw  t3, DIM          # lato della matrice
               la  t4, matrice2D    # ind. matrice2D[0][0]
cicloRighe:    bge t1, t3, fine    # se finite le righe
cicloColonne: bge t0, t3, nextRiga # se finite le colonne
               bne t0, t1, continua # se x != y si continua
               mul t5, t1, t3       # x*N
               add t5, t5, t0       # x*N + y
               slli t5, t5, 2       # word => molt. per 4
               add t5, t4, t5       # calcolo ind.
                                   matrice2D[x][y]
               lw  t5, (t5)         # carico matrice2D[x][y]
               add t2, t5, t2       # e lo accumulo
```

Somma della diagonale (segue)

```
continua:      addi    t0, t0, 1      # y += 1
               j      cicloColonne  # alla colonna successiva
nextRiga:      li      t0, 0         # azzero y
               addi    t1, t1, 1     # x += 1
               j      cicloRighe    # alla riga successiva
fine:          mv      a0, t2        # preparo la stampa
               li      a7, 1         # syscall 1 = print int
               ecall
               li      a7, 10        # syscall 10 = exit
               ecall
```

NOTA: questa è una versione volutamente inefficiente che scandisce tutta la matrice. Può essere resa più efficiente:

- Usando un solo ciclo sulla coordinata x da 0 a DIM-1
- Usando i puntatori (indirizzi in memoria)
- Incrementando il puntatore di DIM+1 elementi = (DIM+1)*4 byte per passare da un elemento della diagonale al successivo

Somma diagonale (EFFICIENTE)

.data

matrice2D: .word 0:400 # matrice di 20x20 word

DIM: .word 20 # lato della matrice quadrata

.text # preparazione degli indirizzi e degli incrementi

main: # t0 = indirizzo dell'elemento corrente

t1 = incremento di una riga + 1 elemento (in byte)

t2 = somma parziale

t3 = indirizzo finale della matrice (byte seguente)

la t0, matrice2D # indirizzo dell'inizio

lw t1, DIM # lato della matrice

*addi t3, zero, 400 # DIM * DIM elementi (al posto di mul)*

*slli t3, t3, 2 # totale DIM^2 * 4 byte*

add t3, t3, t0 # indirizzo finale

addi t1, t1, 1 # DIM + 1

*slli t1, t1, 2 # incremento = (DIM+1)*4*

li t2, 0 # somma iniziale

Somma della diagonale (segue)

```
# t0 = indirizzo dell'elemento corrente
# t1 = incremento di una riga + 1 elemento
# t2 = somma parziale
# t3 = indirizzo finale della matrice (subito dopo)

# ciclo che scandisce un elemento ogni DIM+1
ciclo: bge      t0, t3, fine      # se è finita la matrice esco
      lw       t4, (t0)          # carico matrice2D[x][x]
      add      t2, t4, t2        # e lo accumulo
      add      t0, t0, t1        # x += (DIM+1)*4
      j        ciclo

# stampa del risultato
fine:  mv       a0, t2           # preparo la stampa
      li       a7, 1            # syscall 1 = print int
      ecall

      li       a7, 10           # syscall 10 = stop
      ecall
```

Somma della diagonale (segue)

```
# t0 = indirizzo dell'elemento corrente
# t1 = incremento di una riga + 1 elemento
# t2 = somma parziale
# t3 = indirizzo finale della matrice (subito dopo)

# ciclo che scandisce un elemento ogni DIM+1
ciclo: bge      t0, t3, fine      # se è finita la matrice esco
      lw       t4, (t0)          # carico matrice2D[x][x]
      add      t2, t4, t2        # e lo accumulo
      add      t0, t0, t1        # x += (DIM+1)*4
      j        ciclo

# stampa del risultato
fine:  mv       a0, t2           # preparo la stampa
      li       a7, 1            # syscall 1 = print int
      ecall

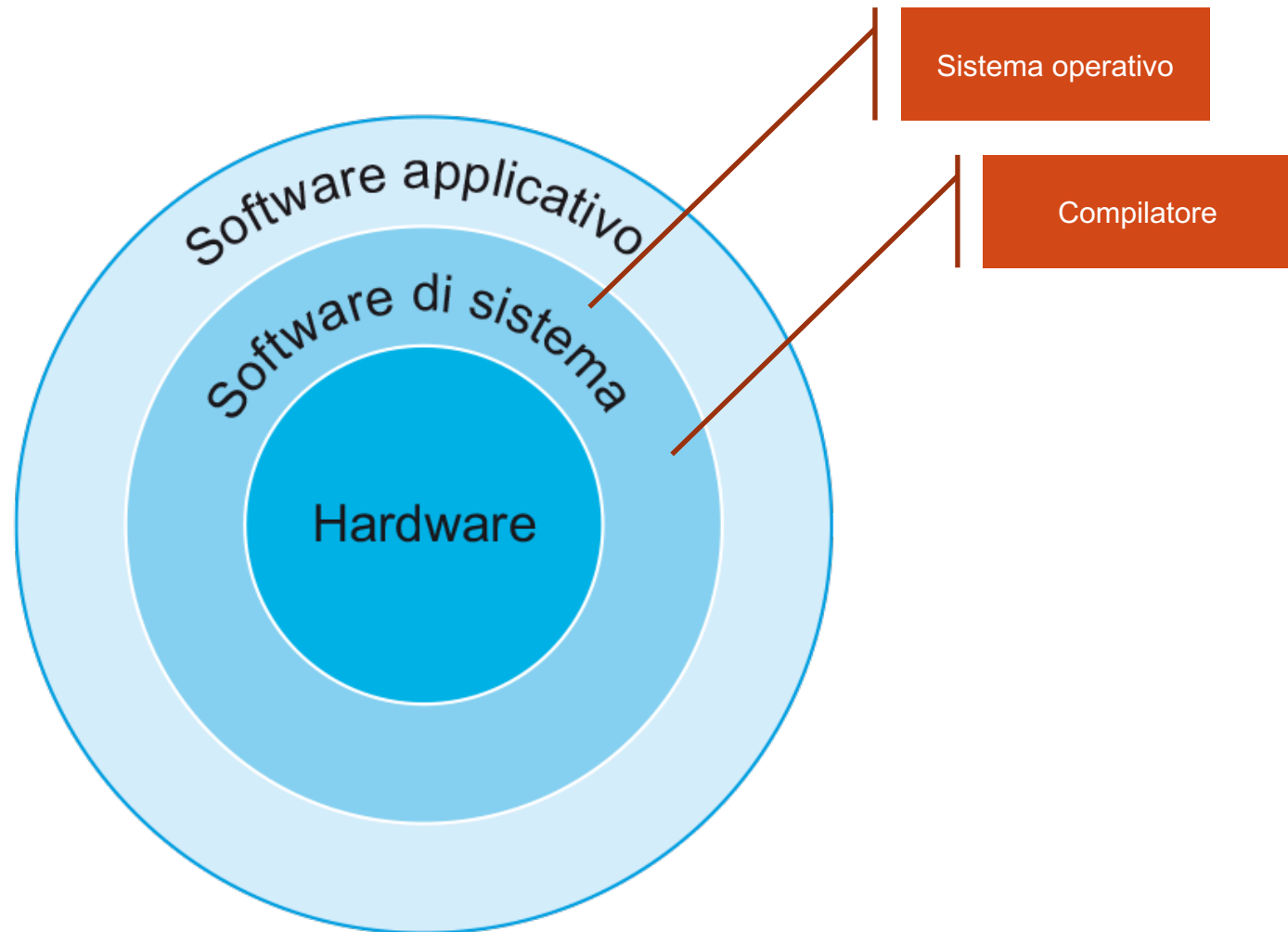
      li       a7, 10           # syscall 10 = stop
      ecall
```

System Calls



SAPIENZA
UNIVERSITÀ DI ROMA

System Calls



syscall

Richieste al sistema operativo

Input:

- a7: operazione **richiesta**
- a0 ... a6: eventuali **parametri**

Output:

- a0, f0: eventuale **risultato**

Syscall (a7)	Descrizione	Argomenti (a0 ...)	Risultato
1	Stampa Intero	Intero	-
4	Stampa Stringa	a0=String Address	-
5	Leggi Intero	a0	a0
8	Leggi Stringa	a0 = buffer address a1 = num chars.	
10	exit		

Lo stavate aspettando

```
cdc08x@cdc08x-Latitude-E5570:~$ echo "Hello world!"  
Hello world!  
cdc08x@cdc08x-Latitude-E5570:~$ █
```

Lo stavate aspettando

```
.globl main
```

```
.data
```

```
string: .asciz "Hello world!"
```

```
.text
```

```
main:
```

```
li a7, 4
```

```
la a0, string
```

```
ecall
```

Esempio in RARS

Sulla pseudoistruzione la

Edit Execute

Text Segment

Bkpt	Address	Code	Basic	Source
☐			addi x17,x0,4	1: .globl main
☐			auipc x10,0x0000fc10	3: .data
☐			addi x10,x10,0xffffffffc	4: string: .asciz "Hello wor...
☐			ecall	6: .text
☐				7: main:
☐	0x00400000	0x00400893	addi x17,x0,4	8: li a7, 4
☐	0x00400004	0x0fc10517	auipc x10,0x0000fc10	9: la a0, string
☐	0x00400008	0xffc50513	addi x10,x10,0xffffffffc	
☐	0x0040000c	0x00000073	ecall	11: ecall

Labels

Label	Address ▲
(global)	
main	0x00400000
AE - 06 - HelloWorld.s	
string	0x10010000

☒ Data ☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	l l e H	o w o	! d l r	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010020	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

indirizzo a 32bit → due istruzioni:
auipc usa immediate da 20bit
addi usa immediate da 12bit

Codici per chiamate al sistema operativo

Name	Number	Description	Inputs	Ouputs
PrintInt	1	Prints an integer	a0 = integer to print	N/A
PrintString	4	Prints a null-terminated string to the console	a0 = the address of the string	N/A
ReadInt	5	Reads an int from input console	N/A	a0 = the int
ReadString	8	Reads a string from the console	a0 = address of input buffer a1 = size of the buffer	N/A
Sbrk	9	Allocate heap memory	a0 = amount of memory in bytes	a0 = address to the allocated block
Exit	10	Exits the program with code 0	N/A	N/A
PrintChar	11	Prints an ascii character	a0 = character to print (only lowest byte is considered)	N/A
ReadChar	12	Reads a character from input console	N/A	a0 = the character or -1 if end of input.
Time	30	Get the current time (milliseconds since 1 January 1970)	N/A	a0 = low order 32 bits a1=high order 32 bits
MidiOut	31	Outputs simulated MIDI tone to sound card (does not wait for sound to end).	See MIDI note below	N/A
Sleep	32	Set the current thread to sleep for a time (not precise)	a0 = time to sleep in milliseconds	N/A
Close	57	Close a file	a0 = the file descriptor to close	N/A
Read	63	Read from a file descriptor into a buffer	a0 = the file descriptor a1 = address of the buffer a2 = maximum length to read	a0 = the length read or -1 if error
Write	64	Write to a filedescriptor from a buffer	a0 = the file descriptor a1 = the buffer address a2 = the length to write	a0 = the number of charcters written
Exit2	93	Exits the program with a code	a0 = the number to exit with	N/A
Open	1024	Opens a file from a path Only supported flags (a1) are read-only (0), write-only (1) and write-append (9). write-only flag creates file if it does not exist, so it is technically write-create. write-append will start writing at end of existing file.	a0 = Null terminated string for the path a1 = flags	a0 = the file decriptor or -1 if an error occurred

Procedure (subroutines) e Funzioni



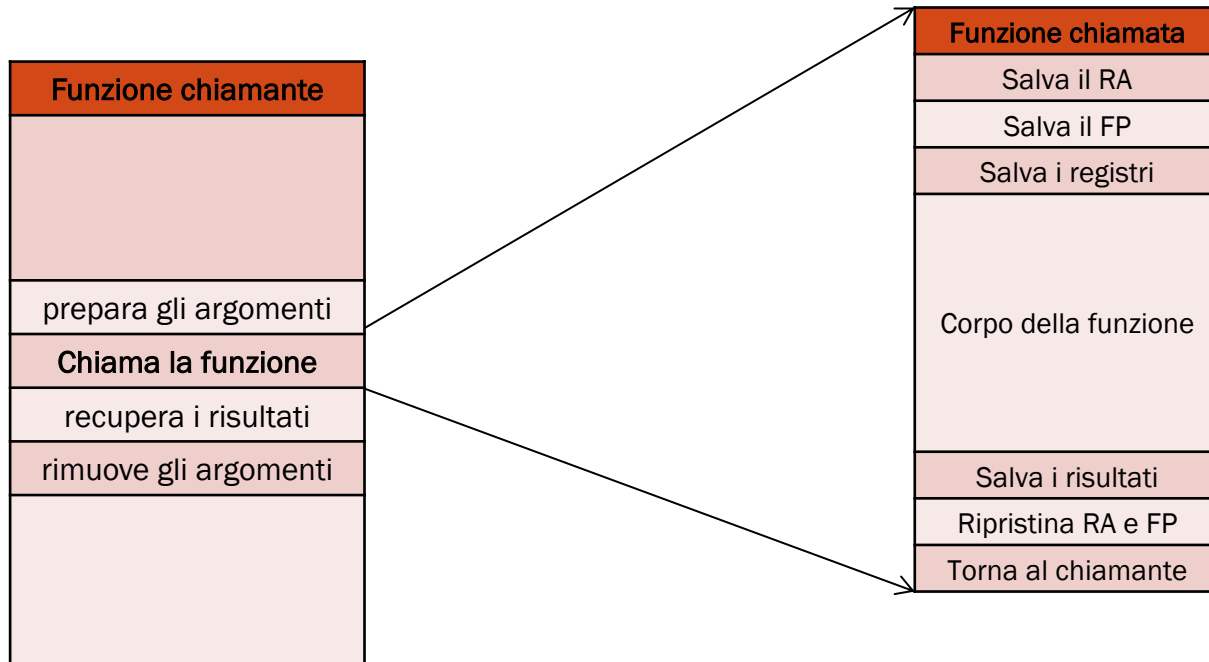
SAPIENZA
UNIVERSITÀ DI ROMA

Procedura e Funzioni

Funzioni / procedure:

Frammento di codice che riceve degli argomenti e calcola un risultato (**utile per rendere il codice riusabile e modulare**)

- ha un indirizzo di partenza
- riceve uno o più argomenti
- svolge un calcolo
- ritorna un risultato
- continua la sua esecuzione dall'istruzione seguente a quella che l'ha chiamata



Funzioni: ingredienti / 1 – Salti incondizionati

Salto
condizionato

```
While: #while (i < N){  
    bge t1, s7, WhileEnd # IF i >= N THEN jump to WhileEnd  
    lw t2, (t1)           #Load Array[i];  
    add s0, s0, t2        # s = s + Array[i];  
    addi t1, t1, 4        # i +=1;  
    j While               #}  
WhileEnd:
```

Salti incondizionati	Salta e collega	jal x1, 100	x1 = PC+4; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	x1 = PC+4; vai a x5+100	Ritorno da procedura; chiamata indiretta



Istruzioni necessarie

Per chiamare la funzione/procedura

jal rd, etichetta Jump And Link

(simile ad un salto a subroutine ma più limitato)

ricorda nel registro **rd** la posizione dell'istruzione successiva ($rd \leftarrow PC+4$)

cambia il PC per iniziare l'esecuzione del corpo della funzione ($PC \leftarrow PC + \text{etichetta}$)

j etichetta → jal x0, etichetta

Per tornare e continuare l'esecuzione del programma chiamante

jalr rd, rs1, offset Jump to Register

ricorda nel registro **rd** la posizione dell'istruzione successiva ($rd \leftarrow PC+4$)

salta all'indirizzo contenuto nel registro indicato ($PC \leftarrow rs1 + \text{offset}$)

jr ra → jalr x0, 0(ra)

Come passare valori ALLA funzione (caso semplice)

a0, a1, ... , a7 8 registri per passare e restituire 8 valori a 32 bit

Come passare valori DALLA funzione (caso semplice)

a0, a1 2 registri per restituire 2 valori a 32 bit

nota (convenzioni):

t0, t1, ... possono cambiare tra una chiamata e l'altra (temporary)

s0, s1, ... non cambiano tra una chiamata e l'altra (saved)

Case Switch Program + System Calls + Subroutines

`.globl main`

main

`.data`

```
switch_case: .word case_00, case_01
case_string: .asciz "Ran case #: "
input_question: .asciz "Input 0 for s
address_of_forbidden: .word forbidden
hidden_text: .asciz "forbidden!"
```

`.text`

`main:`

```
jal read_int # Leggi intero
mv t2,a0 # mettilo in t2

jal read_int # Leggi intero
mv t3,a0 # mettilo in $t3

la a0, input_question # ask question
jal print_string

jal read_int # Leggi caso 0 o 1
mv t1,a0
slli t0, t1, 2

la t4, switch_case
add t4, t4, t0
lw s1, (t4) # load the corrisponding

la a0, case_string
jal print_string # Print first case

mv a0,t1 # Print 0 or 1
jal print_rez
jr s1
```

`case_00:`

```
# sum
add t3,t3,t2
j end_switch
```

`case_01:`

aux

`.globl print_string, read_int, print_rez, print_char, exit`

`.data`

`# Syscall lookup table`

```
print_s: .byte 4
print_i: .byte 1
read_i: .byte 5
print_c: .byte 11
exit_s: .byte 10
```

`.text`

```
print_string: # cenno a una procedura # print a string (4 means
lb a7, print_s
ecall
jr ra
```

`print_rez:`

```
lb a7, print_rez
ecall
jr ra
```

`read_int:`

```
lb a7, read_int
ecall
jr ra
```

`print_char:`

```
lb a7, print_char
ecall
jr ra
```

`exit:`

```
lb a7, exit_s
ecall # Very important: # if we do not ad
```

Settings	Tools	Help
☒		Show Labels Window (symbol table)
☒		Addresses displayed in hexadecimal
☒		Values displayed in hexadecimal
		Assemble file upon opening
		Assemble all files in directory
☒		Assemble all files currently open
		Assembler warnings are considered errors
☒		Initialize Program Counter to global 'main' if defined
		Derive current working directory

Compito per casa (o per qualsiasi altro edificio 😊)



1. Installare **RARS** e prendere familiarità con interfaccia
2. Scrivere i programmi visti a lezione e provare ad eseguirli e debuggarli passo passo
 - Controllare come cambiano i registri sulla destra in base ai passi svolti.
 - Ispezionare come cambiano le pseudo-istruzioni immesse nelle istruzioni che poi svolge veramente il calcolatore
3. Partendo dal programma che trova il max in un vettore scrivere un programma in linguaggio assembly RISC-V con RARS che dato un vettore ingresso **vector** e la sua dimensione **N** calcoli due somme dei numeri del vettore.



1. La prima somma deve sommare i valori del vettore di indice **dispari**. (Indice parte da 1)
2. La seconda somma deve sommare i valori di un vettore con **indice pari**. (Indice parte da 0)

Vector .word 4, -1, 5, 500, 0, 10000, -256

N .word 5

Somme: .word 0, 0

0 1 2 3 4 5

Una volta scritto con questi dati cambiate N e controllate se continua a funzionare

Name	Num...	Value
\$ZERO	0	0
\$at	1	0
\$v0	2	0
\$v1	3	0
\$a0	4	0
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	0
\$t1	9	0
\$t2	10	0
\$t3	11	0
\$t4	12	0
\$t5	13	0
\$t6	14	0
\$t7	15	0
\$s0	16	0
\$s1	17	0
\$s2	18	0
\$s3	19	0
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	268468224
\$sp	29	2147479548
\$fp	30	0
\$ra	31	0
pc		4194384
hi		0
lo		0

Una possibile soluzione

```
.data
#####
# Declaration of constants and variables in memory
#####
Array: .word 4, -1, 5, 500, 0, 10000, -256
N:      .word 7
#Array: .word 1, 2, 2, 4
#N:      .word 4
Sums:   .word 0, 0

.text
#####
# Initialisation and loading
#####
and t1, t1, zero # $t1: cursor Index for the Array
add t2, zero, x0 # $t2: temporary value loaded from Array[$t1]
addi t3, zero, 0 # $t3: a flag for 3ven elements (1 if even), or I could look at the parity of a counter
li t5, 0 # $t5: off5et
andi s3, s3, 0 # $s3: sum of 3ven elements in Array
xor s0, s0, s0 # $s0: sum of 0dd elements in Array

lw s7, N # $s7: the length of Array ($s7 ← N)

#####
# Core business
#####
while:
    bge t1, s7, whileEnd # Exit the cycle if the cursor goes beyond the length of Array
    # {
    or t3, t3, x0 # this plus next: nor
    not t3, t3 # Switch the flag. When $t3 == 0, $t3 ← 11...1. When $t3 == 1, $t3 ← 00...0
    slli t5, t1, 2 # offset = index × 4

    la s10, Array
    add s10, s10, t5
    lw t2, (s10) # load array value at Base + offset (indexed absolute)
    if:
        bnez t3, else # Is $t3 != 0 ? (even)
    then:
        # If not, $t2 contains a value at an odd index
        add s0, s0, t2 # Add the content of Array[$t1] to $s0 (0dd)
        j endif # Exit the if-then-else block
    else:
        # If $t2 contains a value at an even index
        add s3, s3, t2 # Add the content of Array[$t5] to $s3 (3ven)
    endif:
    addi t1, t1, 1 # Increase the index

    j while
    # }
whileEnd:
# Store the results in Sums
la s10, Sums
sw s3, (s10) # Store the sum of 3even elements is Sums[0]
sw s0, 4(s10) # Store the sum of 0dd elements is Sums[1] (again, remember we are considering 4-byte words)
```